

data_loader.py

Raw data loading, schema validation, and the time-based train/validation split

Three small functions that turn the raw gzip-compressed Avazu CSV into a validated, chronologically split pair of DataFrames — the entry point every training script (`trainer.py`, `run_pipeline.py`) starts from.

load_sample **FUNCTION**

```
load_sample(path=RAW_DATA_PATH, n_rows=SAMPLE_N_ROWS, label_col=LABEL_COL) ->
pd.DataFrame
```

Reads the *leading* `n_rows` of the gzip-compressed raw file via `pandas.read_csv(..., n_rows=n_rows)`. This works as a valid sampling strategy specifically because Avazu's `train.gz` is already sorted by `hour`: reading a prefix yields a contiguous, chronologically-ordered slice of the data rather than an arbitrary or biased subset, and never requires decompressing the full ~5.9GB file.

After loading, it raises `ValueError` if the expected label column is missing from the resulting DataFrame's columns — a fast, fail-loud check before any downstream processing begins.

validate_schema **FUNCTION**

```
validate_schema(df, label_col=LABEL_COL) -> None
```

Two checks. First, the label column's set of unique values must be a subset of `{0, 1}` — enforced by comparing the set of observed labels against `{0, 1}` and raising `ValueError` on any other value.

Second, it computes each column's null rate and prints any column with a nonzero rate, sorted descending. For column `c` over `N` rows this is simply the fraction of missing entries:

Null rate per column

$$\text{null_rate}(c) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[x_{i,c} \text{ is null}]$$

time_based_split FUNCTION

```
time_based_split(df, hour_col=HOUR_COL, val_frac=VAL_FRAC) -> (train_df, val_df)
```

Sorts the DataFrame by `hour_col` ascending, then splits it at a single index into an earlier "train" block and a later "validation" block — **no shuffling**. For `N` total rows and a validation fraction `val_frac`, the split index is:

Rows `[0, split_idx)` become `train_df`; rows `[split_idx, N)` become `val_df`. Both are re-indexed from 0.

This is deliberate and load-bearing for the rest of the project: because validation is always strictly later in time than training, this both simulates real deployment (you can only ever train on the past to predict the future) and prevents future-into-past leakage that a random/shuffled split would silently introduce — e.g. a target-encoded feature "knowing" about a category's click rate from rows that, chronologically, haven't happened yet relative to a given training row.

Split index

$$\text{split_idx} = \lfloor N \cdot (1 - \text{val_frac}) \rfloor$$